

ReadRight 2.0: Milestone 3

Updated Prompt Log

AI Systems Used: Claude (Anthropic) · Gemini (Google)

Entry 1 - Designing the graded: false Retry Flow in practice_screen.dart (Claude)	
1. Context	We had updated CloudFallbackAssessor to return graded: false instead of a silent zero score, but the practice screen wasn't doing anything different with that flag yet. It was still showing the zero as a real score. I needed to figure out how the screen should handle the ungraded case specifically, so a network blip doesn't penalize a kid.
2. Prompt Excerpt	"In our Flutter pronunciation practice screen, when Azure is unreachable our fallback assessor now returns AssessmentResult with graded: false and accuracy/fluency/completeness all set to 0. The screen currently treats every result the same regardless of the graded flag. How should the screen behave differently for an ungraded result, what UI should the child see, and should the attempt be saved to progress at all?"
3. AI Summary	Claude recommended checking the graded flag immediately after assessment and branching into two distinct paths. For graded: false, show a neutral 'We could not hear you clearly, please try again' message without displaying any score, and skip saving the attempt to LocalProgressService entirely so the zero doesn't corrupt the student's history. For graded: true, continue with the existing scoring and feedback flow. It also suggested surfacing a soft connectivity warning rather than a scary error so the child doesn't feel like they did something wrong.
4. Human Evaluation	The branching approach was exactly right, the key insight we hadn't spelled out in code yet was that the attempt should not be saved at all when graded is false, not just displayed differently. Claude's specific wording for the child-facing message was too clinical ('We could not hear you clearly'), so we softened it ourselves to 'Hmm, we had trouble hearing that. Try again!'
5. Final Decision	Accepted the core logic: check graded flag, skip saveAttempt when false, show retry prompt instead of score. Rewrote the child-facing copy ourselves. Did not implement the connectivity warning in this milestone. The final practice_screen.dart branches on result.graded before calling _progressService.saveAttempt and routes to a separate retry UI state.
6. Testing / Verification	Tested by temporarily replacing CloudAssessmentService.instance with the fallback assessor in practice_screen.dart, ran the app, recorded a word, and confirmed: no attempt appeared in LocalProgressService after the session, and the retry message was shown instead of a score. Restored the real assessor and confirmed normal scoring continued to work. Also ran cloud_fallback_assessor_test.dart to verify graded: false is returned consistently.

Entry 2 - Syllable Counting Edge Cases in WordTimingService (Gemini)	
1. Context	WordTimingService.syllableCount uses a vowel-group counting heuristic with a silent trailing-e correction. Before writing the unit tests in word_timing_service_test.dart, I wanted to understand what edge cases this approach gets wrong and whether any of them were frequent enough in the Dolch word lists to cause recording windows to be noticeably too short or too long.
2. Prompt Excerpt	"Here is our syllable counting implementation in WordTimingService. It counts vowel groups and subtracts one for a silent trailing e. Which Dolch sight words would this heuristic miscategorize, and are any of those miscategorizations bad enough to make the recording window wrong in practice - either cutting off a child mid-word or waiting too long after they finish?"
3. AI Summary	Gemini identified three categories of miscounts: words ending in '-le' (e.g. 'little', 'apple') where the trailing-e rule is already excluded by the !endsWith('le') guard so those are handled correctly; vowel-heavy words like 'beautiful' where the three-vowel-group count (beau-ti-ful) happens to be right; and short words like 'blue' or 'true' where the trailing-e subtraction brings the count to 1 when the real syllable count is also 1. It concluded that for the Dolch word set specifically - which is dominated by short, simple words - the heuristic is accurate enough and no common Dolch word would get a recording window so wrong as to actively hurt a student.
4. Human Evaluation	We cross-checked Gemini's analysis against the actual Dolch CSVs in lib/data/. The Dolch lists (prek through 3rd grade plus nouns) are indeed mostly one- and two-syllable words with predictable phonics. Gemini's conclusion that no common Dolch word would get a badly wrong window matched our manual check. The test cases we wrote - 'the' (1 syllable), 'beautiful' (3), 'ate' (1, silent e), empty string (1) - cover the main heuristic branches.
5. Final Decision	Accepted Gemini's analysis - no changes to the syllable counting logic were needed. We used its categorization to decide which test cases were worth writing in word_timing_service_test.dart. The four groups in the test file (short words, multi-syllable words, silent-e words, empty input) directly reflect the edge cases Gemini flagged. We did not add a lookup table for irregular words since the Dolch set doesn't have enough irregular cases to justify the complexity.
6. Testing / Verification	Ran flutter test test/word_timing_service_test.dart - all 6 tests pass. Manually verified recordingDurationFor against three representative Dolch words: 'the' (1 syllable → 1800ms min, clamped), 'beautiful' (3 syllables → 3200ms), 'little' (2 syllables → 2500ms). All windows felt natural when we ran the app and timed a child reading each word aloud at a normal pace.

Entry 3 - Content-Safety Guardrail: Fails-Closed Moderation Design in server.js (Claude)	
1. Context	Our server.js calls OpenAI's moderation endpoint both on the teacher's theme input and on the generated story output. I was writing the

	isFlaggedByModeration function and wasn't sure what the right behavior was if the moderation API call itself fails - should we let the content through, or block it? This matters because a classroom app cannot afford to accidentally surface inappropriate content due to a transient API error.
2. Prompt Excerpt	"In our Node.js backend proxy for a children's literacy app, we call OpenAI's moderation endpoint on both the teacher's input theme and the generated story output before sending it to the Flutter app. If the moderation API call itself throws an error or returns a non-200 status, what is the safer default - treat the content as safe and pass it through, or treat it as flagged and block it? What is the standard term for this design choice?"
3. AI Summary	Claude explained that this is a 'fail-closed' vs 'fail-open' design decision. Fail-closed means blocking when uncertain; fail-open means allowing when uncertain. For a children's app where the cost of showing inappropriate content to a child is much higher than the cost of a teacher needing to regenerate a story, fail-closed is the correct choice. It recommended returning true (flagged) from the catch block and logging the error so the team knows the moderation service had a problem, rather than silently passing content through.
4. Human Evaluation	This was the answer we already leaned toward intuitively, but Claude naming the pattern 'fail-closed' and explaining why the asymmetry of costs matters was useful for documenting the design decision in code. The comment in server.js ('Fails closed: if the moderation call itself errors, we treat the text as flagged rather than silently skipping the safety check') came directly from internalizing Claude's explanation, rewritten in our own words.
5. Final Decision	Accepted. isFlaggedByModeration returns true in both the catch block and when the moderation API returns a non-200. The console.error call in both branches gives us visibility if the moderation service is having problems. This matches what is currently in server.js. We independently verified this is consistent with how OpenAI recommends using their moderation endpoint in their documentation.
6. Testing / Verification	Temporarily modified isFlaggedByModeration to use a bad URL so the fetch would throw, then sent a POST /api/story request. Confirmed the server returned 422 instead of passing the content through, and that the error was logged to console. Restored the correct URL and verified normal story generation continued to work. Also checked that both guardrail branches - input theme and output story - both call the same function so neither can fail-open independently.

Entry 4 - FillBlankService: Distractor Selection and Option Shuffling (Gemini)	
1. Context	While writing the unit tests for FillBlankService, I wanted to make sure the distractor selection logic was actually fair - that the distractors were drawn from the same Dolch list as the target word, that the target word always appeared exactly once in the options, and that the shuffling was genuinely random rather

	than always putting the target in the same position. I used Gemini to review the logic before finalizing the test cases.
2. Prompt Excerpt	"Here is our FillBlankService.buildRound method. It selects distractors from the word pool, builds an options list with the target word, then shuffles. Can you spot any edge cases where the target word might appear more than once in the options list, or where the shuffle might produce a predictably ordered result? Also, does the fallback when the word is not in the example sentence work correctly?"
3. AI Summary	Gemini identified one real risk: if the pool happened to contain the target word more than once (e.g. duplicate entries in a CSV), it would appear multiple times in the options because the distractor filter only excludes case-insensitive matches, not duplicate instances. It also noted that the shuffle is seeded by Dart's default Random which is not deterministic between runs, so test cases using Random() directly would be flaky - a seeded Random(n) should be used in tests. The fallback for a word not found in the sentence (appending '____') was correct.
4. Human Evaluation	The duplicate-in-pool observation was valid but not a practical risk for us because our Dolch CSVs are deduplicated at the data level - we checked. The seeded Random point was the more immediately actionable one, and we had already planned to use it in tests but Gemini confirmed why it mattered. The fallback behavior assessment matched what we saw when we manually tested with a word that didn't appear in its own example sentence.
5. Final Decision	Accepted the seeded Random recommendation for tests - all five test cases in fill_blank_service_test.dart use FillBlankService(random: Random(n)) with different seeds. Did not add a deduplication guard to the pool in FillBlankService itself since the data layer is already deduplicated and adding a runtime guard would be complexity for a risk that doesn't exist in our data. Noted the duplicate risk in a code comment for future maintainers.
6. Testing / Verification	Ran flutter test test/fill_blank_service_test.dart - all 5 tests pass consistently across 10 runs with no flakiness. Manually inspected the Dolch CSV files in lib/data/ to confirm no duplicate entries. Tested the fallback branch by calling buildRound with a WordItem whose exampleSentence did not contain the word and confirmed the sentence received '____' appended correctly.

Entry 5 - StoryBuilderScreen: Saving to Firestore via StoryRepository (Claude)	
1. Context	The StoryBuilderScreen generates a story and then offers a Save button. I needed to understand what the correct Firestore document path should be for saving a story so it could later be retrieved per-student, and whether we needed a separate Firestore security rules update to allow the teacher to write to that path.
2. Prompt Excerpt	"In our Flutter app, a teacher generates a story for a specific student and wants to save it. We store data in Firestore under a path structure tied to the teacher's ID and class. Where should a saved story document live in Firestore so it can be retrieved efficiently per student, and does our existing Firestore security rule -

	which allows a teacher read/write to their own data - already cover a write to that path?"
3. AI Summary	Claude suggested a path of teachers/{teacherId}/classes/{classId}/students/{studentId}/stories/{storyId} and pointed out that if our existing rules grant a teacher write access to teachers/{teacherId}/**, the wildcard would cover this nested path without any rules change. It also recommended using a server-generated document ID (Firestore's auto-ID) rather than a client-generated one to avoid collisions, and suggested including a createdAt timestamp field using FieldValue.serverTimestamp() so stories can be sorted by recency.
4. Human Evaluation	The path structure matched what we had already outlined in StoryRepository, which was a good sign. The rules coverage point saved us from making an unnecessary edit to firestore.rules - we checked the existing rules in firestore.rules and confirmed the wildcard does cover the story subcollection. The auto-ID recommendation was already what Firestore's add() method does by default, so no change was needed there. We did add the serverTimestamp field based on Claude's suggestion.
5. Final Decision	Accepted the path structure and serverTimestamp recommendation. No change to firestore.rules was needed - the existing teacher wildcard rule already covers the story subcollection. StoryRepository.saveStory() uses Firestore's add() for auto-ID and includes a createdAt field. The getStoriesForStudent() query orders by createdAt descending so the most recent story appears first in the history list on the screen.
6. Testing / Verification	Ran the app, logged in as a teacher, opened AI Story Builder, selected a student, generated a story, and tapped Save. Opened the Firebase console, navigated to the Firestore database, and confirmed a document appeared under the expected path with the story text, word list, grade level, theme, and createdAt timestamp. Navigated back into the Story Builder for the same student and confirmed the saved story appeared in the Previously saved stories list.

Entry 6 - Handling the gradeLevel Field in Story Generation - Passing It Through the Proxy (Gemini)	
1. Context	The StoryBuilderScreen lets a teacher pick a Dolch reading level from a dropdown (prek, kindergarten, 1st, 2nd, 3rd). I needed to make sure the gradeLevel was being passed all the way from the Flutter UI through StoryService to the backend proxy and into the story prompt, and that the proxy was validating it against the allowed set rather than just passing anything through to OpenAI.
2. Prompt Excerpt	"In our setup, the Flutter app sends a gradeLevel string in the POST /api/story request body. Our server.js validates gradeLevel against a hardcoded set of allowed values. What happens if a teacher's device sends a gradeLevel value that is slightly different from what the server expects - for example, '1st Grade'

	instead of '1st' - and what is the cleanest way to keep the Flutter DolchLevel.key values in sync with the server's ALLOWED_GRADE_LEVELS set without having to update two files every time a level is added?"
3. AI Summary	Gemini identified this as a classic client-server contract mismatch risk. It suggested two approaches: (1) add a normalization step in server.js that strips spaces and lowercases the incoming gradeLevel before validating it, reducing sensitivity to minor formatting differences; or (2) have the server expose a GET /api/levels endpoint that returns the allowed list, so the Flutter app fetches it at startup rather than hardcoding the values. It noted that option 2 is cleaner architecturally but adds a startup network call that isn't worth it for a list this small and stable.
4. Human Evaluation	We went back and checked the actual DolchLevel.key values in dolch_catalog.dart against ALLOWED_GRADE_LEVELS in server.js - they already matched exactly ('prek', 'kindergarten', '1st', '2nd', '3rd'). So the mismatch risk Gemini flagged was real in principle but not currently present in our code. The normalization suggestion (option 1) is a good defensive measure. We did not implement the /api/levels endpoint since the level set is unlikely to change mid-project.
5. Final Decision	Accepted option 1 partially - added a .toLowerCase().trim() normalization to the gradeLevel value in server.js before it hits the ALLOWED_GRADE_LEVELS check, as a lightweight defensive measure. Rejected the /api/levels endpoint as unnecessary complexity for a four-value stable set. Added a comment in both dolch_catalog.dart and server.js noting that the key values must stay in sync, with a reference to ALLOWED_GRADE_LEVELS so a future developer knows where both live.
6. Testing / Verification	Sent a POST /api/story request via curl with gradeLevel set to '1st' (correct), then with '1ST' (uppercase), then with ' 1st ' (trailing space). All three returned a valid story after the normalization step. Then sent '1st Grade' (the most likely accidental mismatch) and confirmed the server returned a 400 with a clear error message. Checked the Flutter app's dropdown values in dolch_catalog.dart to confirm they all match the normalized expected values exactly.